Syntactic and Semantic Considerations for Parallel Compilation



Dawid Sobczak
20273118

Department of Computer Science and Information Systems

University of Limerick

Supervisor: Annette Mc Elligott
Submitted to the University of Limerick
for the degree of BSc (Hons) in Computer Systems

Abstract

A compiler is an indispensible tool in software engineering. Traditional compiler architectures focus on optimising the single threaded compilation performance through langauge design decisions and the software's architecture. Since multi-core processors have become commonly available over the past two decades, fully utilising these resources has become increasingly important. This project aims to research modern parallel complation methods employed in a parallel compiler frontend. Such a compiler is implemented in order to demonstrate the utility of a data-parallel compiler frontend.

Table of Contents

Abstract I
Table of Contents II
List of Figures III
Program Listings IV
Acronyms VI
Glossary VIII

1 Introduction **1**
1.1 Sequential Compilation 2
1.2 Advantages of Parallel Compilation 4
1.2.1 Increased Compilation Speed with Additional Cores . 4
1.2.2 Parallel Compilation Research Aids Language Design 5
1.3 Advantages of Data-Parallel Compilation 5
1.3.1 Parser Error Recovery 5
1.3.2 Applications in Massively Parallel Computing 6
1.3.3 Increased Performance in Individual Compilation Stages 6
1.4 Project Objectives 7
1.5 Structure of Report 7

2 Literature Review **8**
2.1 Parallelization Methods 8
2.1.1 Single Instruction Multiple Data 9
2.1.2 Multithreading/Multiprocessing 9
2.1.3 General Purpose Computing on GPUs 10
2.2 Methods of Parallelizing a Compiler 10
2.3 Lexing 11
2.3.1 Simulation of DFA 12
2.3.2 Speculative Simulation 13
2.3.3 Prescanning 13

2.4 Parsing . . . 14
2.4.1 Parallel LR Parsing . . . 15
2.4.2 Parallel LL Parsing . . . 16
2.5 Semantic Analysis . . . 16
2.6 Conclusion . . . 17

3 Design **19**
3.1 Lexer . . . 19
3.2 Parser . . . 21
3.3 Semantic Analysis . . . 23
3.4 Conclusion . . . 23

4 Implementation **25**
4.1 Structure . . . 25
4.1.1 Lexer Preprocessing . . . 26
4.1.2 Parser Preprocessing . . . 27
4.1.3 Parallelisation . . . 28
4.1.4 Lexical Analysis . . . 28
4.1.5 Syntactic Analysis . . . 30
4.2 Test Language Processing . . . 31
4.3 Online Compiler . . . 32
4.4 Dependencies . . . 32
4.4.1 Core . . . 32
4.4.2 Utility or Minor Contribution . . . 33
4.4.3 Optimisation . . . 33

5 Evaluation **34**

6 Conclusion **36**
6.1 Summary . . . 36
6.2 Concluding Remarks . . . 37

References **44**

List of Figures

1 Compilation phases in a generic compiler 3

2 NFA of the JSON lexical grammar. The empty string, which is typically denoted with ϵ, is denoted here with \0. 29

3 DFA of the JSON lexical grammar 30

4 Visualization of the parse tree on the left with the source code on the right. Terminals are capitalised and nonterminals are in camel case. 31

5 Times Speedup compared to the number of threads used to process 1MB of JSON. 34

List of Listings

1 Factorial in the test language. 26
2 Example of parsable JSON. 27
3 JSON lexical grammar keywords and their corresponding regular expressions 37
4 An operator precedence parsing grammar for JSON. 38
5 An operator precedence parsing grammar for the test language. 43

Acronyms

AMD	Advanced Micro Devices 6
AST	Abstract Syntax Tree 2, 14, 18, 23–26, 32, *Glossary:* abstract syntax tree
CPU	Central Processing Unit 4, 6, 9, 10, 16, 17
CYK	Cocke–Younger–Kasami 16
DFA	Deterministic Finite Automata 11–14, 17, 19, 20, 24, 27, 28
DFT	Depth First Traversal 23
DKY	Doesn't Know Yet 16
DPU	Data Processing Unit 6
FNF	Fischer Normal Form 22
FSM	Finite State Machine 11, 17, 18, *Glossary:* finite state machine
FYP	Final Year Project 1, 7, 26, 36, 37
GPGPU	General Purpose Graphical Processing Unit 10, 36
GPU	Graphics Processing Unit 6, 10, 16, 17
HTML	Hyper Text Markup Language 4
JSON	JavaScript Object Notation 6, 25, 31
LL	Left-to-right, Left-most-derivation 14–16
LR	Left-to-right, Right-most-derivation 15

NFA	Non-deterministic Finite Automata 11, 20, 26, 27
NIC	Network Interface Controller 6
OPG	Operator Precedence Grammar 5, 15, 18, 21, 22, 24, *Glossary:* operator precedence grammar
ROCm	Radeon Open Compute platforM 6
SIMD	Single Instruction Multiple Data 8, 9, 13, 26, 36

Glossary

abstract syntax tree a hierarchical, tree-like data structure that represents the syntactic structure of source code in a programming language, excluding details of the concrete syntax like punctuation or whitespace. Each node in the tree corresponds to a syntactic construct in the source code, such as an expression, statement, or declaration. The tree's structure reflects the grammatical relationships and nesting of these constructs, with parent-child relationships indicating how they are related in the code 2

compiler software that converts a program written in a high-level language into semantically equivalent instructions in a low-level programming language, the resulting instructions can then be read and executed by the computer I, 1

compiler frontend the initial stages of compilation are colloquially referred to as the frontend of a compiler, such stages include tokenisation, parsing and semantic analysis 1, 2

data-parallel in computer processing, data parallelism refers to dividing or partitioning the data amongst multiple processors (aka graphical processing units or nodes), so that multiple threads can operate concurrently (applying the same operation) on each segment, thereby increasing data throughput 5, 6, 10, 11

finite state machine a mathematical model used to represent a system with distinct states and transitions between these states. At any given time, the system is in one of its defined states. Transitions between states occur in response to specific inputs, and these transitions are governed by a set of rules or conditions 11

Huffman encoding an algorithm used to compress information (characters, images, audio and so forth) without losing information (i.e., a lossless data compression algorithm), named after its creator David Huffman in 1952. The algorithm works by identifying frequently occurring items in the information and assigning them a short sequence of bits in the encoding, while using longer sequences of bits for less frequently occurring items 2

linker a software tool that performs the final step of creating an executable program known as linking. This step involves combining object files created by a compiler and resolving external symbol references 2

network packet parsing an algorithm that reads and analyzes network packets in order to decide how they should be processed in a network-connected device. A network packet is a formatted unit of data carried throughout a computer network 2

operator precedence grammar a type of formal language grammar. It is a context-free grammar that has the property (among others) that no production has either an empty right-hand side or two adjacent nonterminals in its right-hand side 5

regular expression a sequence of characters that specifies a match pattern in text 11, 19, 21

rustc the first and most popular compiler for the Rust programming language that is being developed by the Rust Foundation 1

Chapter 1: Introduction

A compiler is an important part of the interface between a programmer and the underlying hardware of a computer. The modern software ecosystem could not exist without the frequent and extensive use of compilers in the software development process. It is of no surprise that the study of compilers and programming languages has been an important field of research in computer science for many decades. Any improvements in compiler technology can have wide and long lasting affects on software development as a whole (Hall et al., 2009).

The focus of my Final Year Project (FYP) is to study a subset of compiler research techniques concerning aspects of parallel compilation. Research in this niche has historically been of low priority. Traditional compiler architecture since the 1970s has focused on minimizing memory use and optimizing for single threaded performance. This was at a time when computers did not even have enough random access memory to store all the data necessary for compiling large amounts of source code. Commodity hardware has improved significantly since then. The memory available in a computer is much more abundant than before and single threaded processor speed is not improving at the same rate as it once did (Shalf, 2020; Williams, 2017). Nowadays, in fact, we are seeing commodity processors comprising of several cores capable of running dozens of processes concurrently. I believe this is sufficient cause to consider alternative architectures that more efficiently utilize this environment. A parallel compiler utilizes these methods of executing code in parallel in order to speed up the compilation process.

Modern compilers in use today implement varying amounts of parallelism in order to speed up compilation times. The rustc compiler team reports a 50% performance improvement after making parts of the compiler frontend work in parallel (Nicholas Nethercote, 2023). Such improvements are not limited

to the compiler frontend. The mold linker project boasts significantly better performance than its competitors due to its extensive and clever use of parallelism (Rui Ueyama, n.d.). The potential for performance improvements is evident in modern compilers and tools. Moreover, compiler research and development can generalize to other areas outside of software development.

Applications of techniques traditionally associated with compilers can be found in network packet parsing (Wang et al., 2019; Roesch, 1999) and Huffman encoding in lossless image compression (Howard and Vitter, 1996). These areas, among others, can benefit from research directed at improving compiler technology due to a significant overlap in the underlying theory (Mytkowicz et al., 2014). Futher development in compiler theory can benefit not only software development but also tangential fields of research.

Before delving any further into parallel compilation, Section 1.1 will describe compilation in general, paying particular reference to the traditional method of sequential compilation.

1.1 Sequential Compilation

Compilation is the process of translating a program written in a high-level programming language into a semantically equivalent program written in a lower-level programming language such as assembly or machine language. During this process, a compiler will usually lose information in the source code that is useful to the programmer but is useless to the machine that will ultimately execute the program. Inside a compiler, the source code passes through several algorithms that transform the code in various ways. These compilation stages are shown in Figure 1.

The work of a compiler can be described as a multi-stage process resembling a pipeline, where each stage works on the output of the previous stage thereby producing the input for the next stage. The lexer splits the source code into lexemes which are then tokenized for the next stage. In the parser stage, based on the rules of the source language, the code's syntax is verified. During this stage, an Abstract Syntax Tree (AST) is created, reflecting the hierarchy of the elements in the code such as, tokens, variables and statements, and the relationships between them. This AST is then analysed during the semantic analysis stage, considering type checking, variable declaration prior to usage, unused variables and so forth. The parts

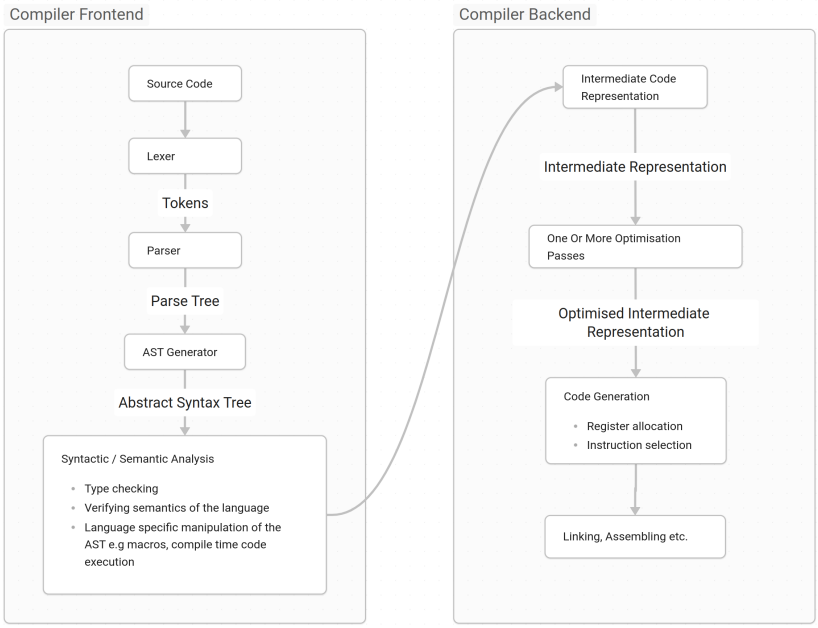


Figure 1: Compilation phases in a generic compiler

of the compiler up to this point are colloquially called the frontend of the compiler.

These stages of compilation are often executed sequentially with each stage doing the least amount of work necessary before passing control to the subsequent stage. For example, the lexer will do only enough work to retrieve the next token before passing that token to the parser. The parser will then begin executing, doing as much work as it can with the input it got from the lexer. Once it has done that, it might request more tokens from the lexer, in which case control is returned to the lexer. This process repeats until the entire contents of the source code in the input file has been processed. Compiling in this way is much more memory efficient compared to performing a compilation stage to completion prior to moving onto the next stage. Difficulties with implementing this architecture using parallel processing methods is described in Section 2.2.

This type of architecture has traditionally been driven by a need to optimize for memory use and single threaded performance. For example, in the C programming language, programmers are forced to declare functions before they are called. This was originally done to allow the compiler to fully compile each function independently, without needing to keep all the other functions in memory and avoiding a costly second pass over code (Scott, 2015). As a result of these decisions as well as historical circumstances, the stages of a compiler that are responsible for taking a file containing source code to code generation are often implemented with little to no parallelization. An example of going from source code to machine code is the process of compiling a C file into an object file. As can be seen from Figure 1, this envelopes a significant portion of the compiler. The next section describes parallel compilation.

1.2 Advantages of Parallel Compilation

Parallel compilation uses parallel processing methods in order to perform various compilation stages at the same time. Parallel processing involves executing more than one piece of code simultaneously. This can be achieved in various ways depending on the available hardware. The most common of these methods are reviewed in Section 2.1. Potential ways to structure a compiler that uses such methods is discussed in Section 2.2. This section considers the motivation for performing various stages of compilation in parallel.

1.2.1 Increased Compilation Speed with Additional Cores

Using multiple Central Processing Unit (CPU) cores during compilation can significantly speed up compilation time. IBM researchers created an optimized lexer for a machine that had a large number of cores that could run code in parallel (Scarpazza and Russell, 2009). It had eight CPU cores where each core could run eight threads in parallel for a total of 64 parallel threads. More recent research into parallel lexing from 2014 achieved a 14 times speed up for lexing Hyper Text Markup Language on a 16 core system (Mytkowicz et al., 2014). Substantial performance gains can clearly be achieved through parallelization. This shows how using parallel processing to speed up compilation tasks is not a new idea.

1.2.2 Parallel Compilation Research Aids Language Design

For several programming languages, once their syntax goes into general use, it becomes nearly impossible to reverse or change it, without breaking programs already written in the particular language. This leads to programming languages becoming complicated and difficult to define as new features are added. For example, languages like C++ are very difficult to compile sequentially, let alone in a parallel manner (*Most vexing parse*, 2022; *Difficulties improving C++*, n.d.). Research into and the development of parallel compilers can aid future developers when they are choosing their syntax or what features they want to implement without making their language difficult to compile in parallel. Barenghi et al. (2015) demonstrate how this is possible, by restricting the grammar of a language to an Operator Precedence Grammar (OPG). It is currently rather difficult to design a language that can easily be compiled in parallel due to a lack of research and prior work as compared to sequential approaches.

1.3 Advantages of Data-Parallel Compilation

Several current compilers can operate in parallel by compiling several source code files at the same time. In this work I am looking at methods employing data-parallel techniques where various stages of a compiler can be executed in parallel when compiling the same file. Due to a lack of prior work comparing the two approaches directly, it is difficult to make an objective statement regarding which approach is better. I will instead focus on the potential advantages and disadvantages of using such a fine-grained approach.

1.3.1 Parser Error Recovery

An important responsibility of a compiler is to detect syntax errors and present useful error messages to the programmer. Such syntax errors are often detected during the parsing stage. An issue arises when a parser must then try to continue parsing after encountering a syntax error. This parser recovery process is frequently complicated and suboptimal (Medeiros and Mascarenhas, 2018; Hutchison, 2020). Certain parallel parsing algorithms, such as those described in Clarke and Barnard (1993) and Barenghi et al. (2015), allow a compiler to forgo a complicated parsing recovery process by

being able to restart the parsing process immediately after encountering an error.

1.3.2 Applications in Massively Parallel Computing

There have been increasingly larger investments in the hardware industry for specialized hardware beyond the typical CPU. For example, smart Network Interface Controllers (NICs) that offload network packet parsing from the CPU or Data Processing Unit (DPU) to provide more facilities for networking and input/output related tasks than a typical CPU. A recent article from October 2023 outlined how the software stack of Advanced Micro Devices (AMD) for general purpose computing on graphics cards known as Radeon Open Compute platforM (ROCm) is now among their top priorities (Ward-Foxton, 2023). As time goes on, we are likely to see increasingly specialized and heterogenous hardware (Stefan, 2021).

Current compilation algorithms scale poorly on massively parallel hardware such as Graphics Processing Units (GPUs). This is due to the restrictions imposed on programs by computing in such an environment. An approach utilizing data-parallel data structures described by Hillis and Steele (1986) was applied by Voetter et al. (2022) in order to create a parallel compiler implementation that could run on a GPU. This shows how re-architecting the compilation process makes it possible to execute a compiler in a massively parallel environment, potentially achieving a linear performance increase that scales with the number of available processing cores. This can only be achieved by efficiently using data-parallel techniques.

1.3.3 Increased Performance in Individual Compilation Stages

Data parallelism can massively improve the performance of various workloads that utilise individual compilation stages. Tasks that involve processing large amounts of data benefit the most. Barenghi et al. (2015) demonstrated how JavaScript Object Notation (JSON) file parsing can be performed much faster using a parallel parser. Mytkowicz et al. (2014) optimized a parallel regular expression matching algorithm which has a large number of uses beyond lexical analysis.

1.4 Project Objectives

The goals of my FYP are as follows:

1. Analyse existing parallel compilation techniques for a compiler frontend. I intend to carry out a literature review of recent approaches in order to better understand the best currently available techniques.
2. I will choose a compiler frontend design, implement as much of it as time allows and evaluate its performance. The compiler frontend will consist of the lexing, parsing and semantic analysis stages.

1.5 Structure of Report

Chapter 1 provided an introduction to compilers as well as reasons to study parallel compilation methods.

Chapter 2 describes approaches for compiling in parallel.

Chapter 3 will elaborate on the issues associated with designing a parallel compiler, presenting suggested approaches from literature to overcome these, prior to focusing on the compiler design and implementation decisions specific to this work.

Chapter 4 explains the details of my implementation, referring to aspects such as external code dependancies and program structure.

Chapter 5 describes my testing methodologies and the evaluation of my compiler's performance.

Chapter 6 concludes the report with a summary of the report and concluding remarks.

Chapter 2: Literature Review

In this literature review I discuss techniques for creating a parallel compiler. This review has helped me to develop a better understanding of how parallel compilers work and how to approach implementing one. Time constraints make it impossible to review all literature on the topic however an attempt is made to cover important developments in parallel compilation research.

Section 2.1 describes hardware capable of executing code in parallel.

Section 2.2 describes different ways of designing a compiler such that parts of it can be executed simultaneously.

Section 2.3 describes various algorithms to perform lexing in parallel.

Section 2.4 describes various algorithms to perform parsing in parallel.

2.1 Parallelization Methods

This section describes various hardware solutions for executing code in parallel. Parallel processing methods can strongly impact the performance for a parallel program as well as the complexity of its implementation. This makes it worthwhile to research and analyse various forms of parallelization in order to make informed decisions about how they will be used in the final implementation.

Section 2.1.1 describes Single Instruction Multiple Data (SIMD) instructions which are used to implement instruction-level parallelism.

Section 2.1.2 describes a more granular form of parallelization called multi-threading or multiprocessing.

Section 2.1.3 describes highly parallel computing on graphics cards.

2.1.1 Single Instruction Multiple Data

SIMD instructions are assembly instructions that can process more data than a typical assembly instruction. Instead of processing data with normal registers, SIMD instructions use special registers that are several times larger than normal registers. This allows a processor to compute an assembly instruction over parts of the register in parallel. These instructions are also known as vector instructions since they process lists or vectors of data. As an example, in the Advanced RISC Machines (ARM) instruction set, the ADD instructions can be contrasted with its corresponding vector instruction UQADD8 which adds four 8-bit numbers with four other 8-bit numbers (*ARM Architecture Reference Manual Thumb-2 Supplement*, n.d.).

Since vector instructions are functionally similar to a collection of assembly instructions being executed simultaneously, a program written using vector instructions can be similar in structure to an equivalent sequential one. This makes it possible for compilers to automatically utilize these instructions to speed up existing sequential programs in a process called vectorization. Vector instructions can also be manually written by the programmer in order to improve performance (Langdale and Lemire, 2019; Mytkowicz et al., 2014). Although using vector instructions can be significantly faster than the sequential case, this performance benefit is only up to a point. Increasing performance past a certain threshold requires larger registers which take a long time to architect and standardize in a CPU. The methods described in Section 2.1.2 scale better at the cost of having a more complex implementation.

2.1.2 Multithreading/Multiprocessing

Multicore parallelism makes it possible to have multiple programs execute at the same time through the multi-threading facilities exposed by most modern CPUs. Each process can have its own call stack and its own view of memory. This freedom in program execution and memory access allows for performant designs of complex parallel algorithms. The performance of a well implemented program can potentially scale with the number of CPU cores available to the program. For example, if a parallel program

can run in one second on one thread then in it should run in a quarter of a second on four threads. In reality, there is overhead from thread creation, synchronization and management that prevents a perfectly linear performance improvement.

The downside of multithreading is the difficulty in creating reliable, bug free programs. This processing model adds new classes of bugs, such as data races that can occur in shared memory scenarios. When converting a sequential program to a multithreaded one, it is usually necessary to re-architect significant portions of it in order to better fit this processing model. Furthermore, managing shared memory is difficult and can lead to subtle yet complex bugs.

2.1.3 General Purpose Computing on GPUs

It is possible to use graphics cards for general purpose computing, called General Purpose Graphical Processing Units (GPGPUs). It is conceptually like programming a specialized CPU that has hundreds if not thousands of cores. Programming for a GPU requires a programmer to significantly re-architect a program in order to fit the GPU programming model. There are additional issues, such as the time it takes to compile GPU programs and the large latency between a GPU and main memory. In fact, this latency can be so large that processing small amounts of data can be much less performant than a single-threaded CPU implementation. In general, large scale computing on a GPU can be strongly worthwhile if the program can operate within the constraints of a GPU environment.

2.2 Methods of Parallelizing a Compiler

Gross et al. (1989) outline three key approaches for parallelizing a compiler. The first approach consists of splitting the source code into chunks and processing each chunk individually. Many compilers divide source code at the filesystem-level where each file of source code can be compiled separately from other parts of the program. Although this approach is already somewhat data-parallel, it can be taken even further where each file is split into even smaller chunks, with each of these chunks being processed independently.

The second approach is termed computation partitioning. This is described as a series of sequential computations that can be divided up and processed

in parallel with the results of each computation being joined together at the end.

The final method is pipeling which involves dividing a computation into some number of phases where each phase depends on the output from the previous phase.

Mark Thierry Vandevoorde (1988) attempts to implement a parallel C compiler using a data-parallel approach with a traditional two-pass compiler architecture.

2.3 Lexing

Scott (2015) describes lexing as the process of removing unnecessary characters from source code and grouping characters of interest together into lexemes. This compilation stage emits a list of lexemes that are later passed to the parser. The purpose of lexing is to simplify the task of the parser by reducing the size of its input. Lexing is sometimes called tokenization or lexical analysis and it is performed by a lexer, sometimes called a tokenizer or lexical analyser. Lexers are defined using regular expressions that describe the character sequences which need to be found and emitted from the lexer as lexemes. Since regular expressions can be described as a Finite State Machine (FSM), a lexer can similarly be described as one.

A FSM is a model of an abstract machine that can be in one of a finite number of states at any given time. A FSM works by accepting input symbols which can cause the state of the FSM to change according to a set of state transition rules that define the FSM. A FSM can be described as either a Non-deterministic Finite Automata (NFA) or as a Deterministic Finite Automata (DFA). In a parallel lexer where the output in one thread may depend on the output of another thread, it is undesirable to have different results based on the same input because it can lead to bugs and an unpredictable output. This situation can occur in a NFA which makes it non-trivial to design a parallel lexer around a NFA. The deterministic property of a DFA ensures that the resulting state of a DFA will always be the same for a given input. As such, structuring a lexer as a DFA can ensure a consistent and repeatable computation for a given chunk of source code.

A common approach to multithreaded lexing involves structuring the lexer as a DFA, splitting the input into some number of parts, lexing each part on its own thread and joining up the results at the end. Due to the inherent data dependency between state transitions in a DFA, the initial state for a chunk of code depends on the output computed from lexing the code just before it. This causes the issue of determining the initial start state of all the lexers besides the one computing the first part of the overall input. A solution to this problem is through speculative simulation where an algorithm speculates on the unkown initial states of these DFAs.

2.3.1 Simulation of DFA

One approach to speculative simulation is the brute force method of computing a DFA for every possible state it can be initialized with. Such a lexer begins by splitting its input into chunks and processing each chunk independently on a separate thread. Once all threads are finished lexing, the outputs from each thread are checked, starting from the first chunk of code, such that the finishing state of the lexer corresponds to the initial state of the lexer that computed the subsequent chunk of code. The correctness of computing a DFA in this way relies on the associativity of state transition functions as described in the parallel prefix sum algorithm in Hillis and Steele (1986). This algorithm is explained more formally with both a message passing and a shared memory implementation by Holub and Štekr (2009) and a variation of it in a cloud computing environment is shown by Ko et al. (2012). The need to compute the output for every possible initial state can result in many unnecessary computations which can significantly impact performance for DFAs with many states.

Barenghi et al. (2015) optimize this by using a heuristic where the source code is split into chunks that start with symbols that have known and consistent initial states. These symbols are found by scanning ahead when the source code is being split until such a symbol is found. The symbols that a chunk of code can start with are language dependant. If a symbol is allowed to appear in a lexeme, such as a string or comment, then multiple initial states must be additionally computed.

Instead of using a language specific heuristic, Mytkowicz et al. (2014) attempted to optimize speculative simulation for any given DFA. The core algorithm is very similar to the one previously described. It begins with the assumption of needing to compute the DFA for every possible initial state input. Instead of using a language specific heuristic like Barenghi

et al. (2015), which is determined ahead of time, Mytkowicz et al. (2014) define a convergence algorithm which reduces the number of states that need to be computed at runtime. It follows from the observation that many states transition to the same state on a given input symbol. Once this convergence of state occurs, looking up the state transition for each state becomes redundant as it is going to be the same for subsequent input symbols. By factoring out these common states, the number of actual state transitions that need to be computed drops significantly for most DFAs, especially structured and non-adversarial ones. Heuristics are used to determine when to check for a convergence of states. The previous heuristic as well as this convergence algorithm are similar in some ways however the solution proposed by Mytkowicz et al. (2014) is more generally applicable by not being tied to a specific language.

Mytkowicz et al. (2014) additionally utilize SIMD instructions in order to perform the state transitions for a given transition function and set of states. This parallelizes an important part of the algorithm and significantly improves performance. A technique called range coalescing is also used to reduce the number of memory addresses accessed when computing state transitions for many states. Zhao and Shen (2015) implement a convergence algorthim similar to the one described by Mytkowicz et al. (2014).

2.3.2 Speculative Simulation

Another method is speculative simulation which attempts to simply guess the initial state of the DFAs, possibly with a way to back track or validate its result in case of an incorrect guess. Luchaup et al. (2009, 2011) implement a speculative method which simply guesses the unknown initial DFA state. This optimization works due to its specifc use in intrusion detection systems where a DFA spends most of its time in a small number of states which can be guessed with sufficient accuracy to outweigh the cost of validation in case of failure.

2.3.3 Prescanning

Bernecky (2003) describes a multi-pass parallel APL tokenizer written in APL which performs several scanning phases in order to pre-process source code as well as find strings, comments and identifiers, among other tokens, in distinct passes over the source code. It is not table driven and is instead specialized for APL. Since the computational complexity of the compiler

is high and its performance is not evaluated by the authors, it remains unknown if it is an improved approach to parallel lexing.

A variation of the heuristic approach by Barenghi et al. (2015) is implemented by Li et al. (2021) that generalizes it and makes it language independent. It builds on work from Sin'ya et al. (2013) and Zhao and Shen (2015) to perform a pre-scanning phase by generating and executing a DFA based on the lexical grammar that determines the context for DFAs in a subsequent tokenization phase.

2.4 Parsing

Parsing is the second canonical stage of a compiler. The goal of a parser is to take a list of lexemes from a lexer and structure them as a graph (Scott, 2015). This structure makes it easier to navigate and manipulate the source code using concepts defined by the language like variables, functions and control structures. The graph structure created by a parser is typically called a syntax tree or a parse tree. It is created according to a parsing grammar that defines how it should be built. The rules of a parsing grammar that govern how lexemes should be processed in order to validate an instance of the language and build a parse tree representing it are called production rules. A parser might additionally process a parse tree as it is getting built, or during a separate subsequent processing step, which produces an AST. This data structure is later analysed during the semantic analysis phase. There exist different families of parsers that group parsers by similarities in their method of parsing.

Left-to-right, Left-most-derivation (LL) parsers read the source code from left to right and construct a parse tree from the root node down, predicting at each step which production rule will be used to expand the current node, based on the next available token of input. LL parsers are also called recursive descent parsers due to their typical implementation of recursively calling a function for every non terminal in a production. Many modern compilers use recursive descent parsers because they can easily be implemented by hand and make it is easier to generate good error messages for the user. An alternative approach is to create an LL parse table for a given grammar and use a driver program that parses tokens according to the rules stored in the parsing table. A program that uses this table to generate and output source code for a recursive descent parser is a parser generator.

In Left-to-right, Right-most-derivation (LR) parsing, source code is read left to right but the parse tree is constructed by first creating the leaf nodes and later grouping these nodes together into trees. This kind of parser works by maintaining a stack of tokens and parse tree nodes during parsing. The parser will match this list against the right side of the grammar's production rules. If a match occurs, the tokens on the stack become children of a new node corresponding the left side of the rule. This new node, corresponding to a subtree of the final parse tree, is pushed onto the stack. Parsing finishes once the root node (axiom) is built from the nodes on the stack. This type of parser is also called a shift-reduce parser due to its two key operations, shifting tokens onto the stack and reducing them into new parse tree nodes.

LL(k) and LR(k) denotes how many tokens these parsers must look ahead in the input in order to resolve ambiguities between production rules.

2.4.1 Parallel LR Parsing

Barenghi et al. (2015) proposed and implemented a non-speculative LR (0) parallel parser generator for OPGs. A key insight is in constraining the grammar sufficiently such that it can be deterministically decided whether a string of bounded length contains the right-hand side of a production and can be unequivocally replaced by its corresponding left-hand side. This is in contrast to other approaches that require the parser to speculate on possible production rules in order to reduce them when parsing in parallel (Mickunas and Schell, 1978).

Li and Taura (2023) built on the work by Barenghi et al. (2015) and optimized the parser for a common case encountered in real word data. When parsing a long list described by a recursive production rule, the whole list must be parsed in order to reduce it into a parse tree node. This means that if the list is large enough to be parsed by multiple threads then the final operation which reduces the stack will be deferred until all the threads have finished. Moreover, if there is little work for the parser to do per element in the list, then the bulk of the parsing will be left until the final joining phase of the parser. The contribution from Li and Taura (2023) is a way of recognizing operators in the grammar that are associative so that elements of such lists can be more effectively reduced into parse tree nodes before knowing they are part of a list.

2.4.2 Parallel LL Parsing

In Vagner and Melichar (2007), a deterministic parallel LL parsing algorithm is presented. The algorithm is based on a transformation from a parsing problem to parallel reduction. First, a nondeterministic version of a parallel LL parser is introduced. Then, it is transformed into the deterministic version — the LLP parser. The deterministic LLP(q, k) parser uses two kinds of information to select the next operation — a looka head string of length up to k symbols and a lookback string of length up to q symbols. Deterministic parsing is available for LLP grammars, a subclass of LL grammars.

Mark Thierry Vandevoorde (1988) exploits C's syntax and semantics by forking his recursive descent parser (an LL parser) in places where the code under parse does not depend on the code surrounding it.

Skrzypczak (2012) developed a CPU and GPU implementation of the Cocke–Younger–Kasami (CYK) parsing algorithm.

2.5 Semantic Analysis

Semantic analysis is the third canonical stage of compilation. The goal of this stage is to analyse the output from the parser to see whether it conforms to the semantic rules of the language. Informally, parsing attempts to find the structure of a program whereas semantic analysis tries to find its meaning.

Seshadri and Wortman (1991) implement a parallel semantic analyser for a regular, block structured language. It checks whether variables have been declared before being used. The Doesn't Know Yet (DKY) problem is introduced which describes certain issues with analysing code without the context surrounding the code. An example of the DKY problem is encountered when the analysis process is performed within the code and it isn't yet known whether a variable has been declared. The authors propose three approaches to avoid or minimize the impact of this problem.

1. DKY Avoidance - involves scheduling processes in the compiler in order to avoid DKYs during semantic analysis. This technique was also used by Gross et al. (1989).
2. DKY Handling
3. Two part DKY Handling

2.6 Conclusion

After conducting this literature review, I have gained a broader understanding of the issues involved in parallel compilation and the potential solutions posed by various authors. This experience has been invaluable in designing and implementing my own frontend. This section discusses my findings and considerations for the design of my compiler frontend.

When designing a parallel compiler, one must first choose an appropriate paralleization method. This decision is important because it can impact the performance of the compiler, as well as the programming language and algorithms used in its implementation. For this project I considered between using the eral context of the source code under analysis. I intend to base my own algorithm off the findings by Seshadri and Wortman (1991) parallel facilities of GPUs, multicore CPU and SIMD capable CPU.

Computing on the GPU affords the highest peak performance due to being built with data-parallelism in mind. Unfortunately, however, programming on a GPU is difficult, running small tasks on it is impractical and trying to create a compiler that runs on the GPU requires making big concessions in regard to the compiler architecture and language design. Although promising results were shown by Skrzypczak (2012), even for input as small as 200 lexemes, other work such as that by Voetter et al. (2022) shows very large overhead for average sized inputs. Due to these issues I determined that writing a parallel compiler frontend to work on the GPU would be impractical.

In contrast to the GPU programming, multicore parallelism is more commonly available, flexible and scalable. Although the peak performance of a GPU is higher than a CPU for specific workloads, I believe the benefits of using multicore parallelism outweigh the benefits of GPU paralleism when taking into account the issues associated with GPU programming. After choosing a parallelization method, such as multicore parallelism, it is necessary to choose an appropriate lexing algorithm.

The core issue with parallel lexing is figuring out the initial state of the DFA for all but the DFA processing the first chunk of input. This can be solved enumerating through all possible initial states or simply guessing the most likely state and backtracking if guessed incorrectly. Speculative approaches that guess the initial FSM states can be effective, as shown by Luchaup et al. (2011), however Mytkowicz et al. (2014) point out two major issues that can arise. The efficacy of the speculative approach is difficult to

predict and it is limited by the sequential implementation on a single core. In contrast, enumerating through all possible initial states is predictable against even randomly generated input and can benefit from fine-grained parallelism afforded by a single core.

For my implementation I've chosen to use the enumerative approach where I will enumerate over all possible initial states for the FSMs. I hope this will lead to a more robust system that is resistant to edge cases and adversarial input. I hope to create a table driven lexer which will help me in quickly iterating over the lexical grammar to better fit the parsing grammar.

There are many parser designs available in the literature. After considering the expressiveness of the parsing grammar, the complexity of the implementation and performance results of each parser design, I have chosen the OPG parser by Barenghi et al. (2015). It has a simple design that allows for a straightforward implementation while remaining with an expressive enough parsing grammar for implementing fully-featured programming languages.

The semantic analysis stage requires parallel traversing of the AST, possibly with some amount of synchronization between threads in order to fill in information e.g. a thread may be missing about the general context of the source code under analysis. I intend to base my own algorithm off the findings by Seshadri and Wortman (1991), as described in Section 2.5.

Chapter 3 will describe the design of the implementation based on what I learned from the literature review.

Chapter 3: Design

The design of my compiler is based on the ideas and techniques I encountered during my literature review. Instead of creating a compiler for a specific language instance, I intend to implement a table driven lexer and parser that will make it easy to implement different kinds of languages. The semantic analysis phase cannot easily be made generic and will require a distinct implementation per language. I believe this approach will help me evaluate the impact of parallel processing on compilation by testing not only my own bespoke language but also common languages such as JSON or Lua.

Section 3.1 describes the design of the table driven lexer. This includes my method of defining lexical grammars, the algorithms used in pre-processing the lexical grammar and the algorithms used during the parallel lexical analysis.

Section 3.2 describes the design of the table driven parser. This includes my method of defining the parsing grammar, its limitations, the transformation of the grammar into a bounded context grammar and the parallel parsing algorithm.

Section 3.3 describes the general approach for designing the semantic analyser.

3.1 Lexer

Lexers are defined using regular expressions that describe the character sequences which need to be found and emitted from the lexer as lexemes. These regular expressions can be used to construct a DFA that accepts the source code as input and is capable of recognizing whether the input matches any of the original regular expressions. An example of such a

lexical grammar for JSON is attached in Section 3. In order to construct the DFA, it is necessary to convert the regular expressions into an NFA using Thompson's construction algorithm (Aho et al., 2006), which results in a graph where nodes represent the states and the edges represent state transitions within the NFA. This NFA can then be converted into a DFA through the powerset construction algorithm (*Powerset construction*, 2023).

During this conversion, it is necessary to look for duplicate edges where the input symbols are the same, but the states being transitioned to differ. This process enumerates every potential start state of the lexer and solves the issue posed in Section 2.3. These states are recorded and used during the parallel lexing algorithm.

Sequential lexical analysis can be now be performed by iterating over the input text, starting at the DFAs initial state and following the edges that correspond to each input symbol. If an edge is found, the state changes accordingly and the symbol is discarded. This continues until a node is reached where no edge can be found for the current input symbol. If such a node represents a terminal of the grammar, that terminal is emitted by the lexer. If it doesn't, the lexer emits an error. In either case, the lexer's state is reset to its initial state and the input symbol is processed again instead of being discarded.

Although it is possible to use the DFA directly to perform lexical analysis, it is more efficient to keep track of the current state in a variable and compute the state transition by using a lookup table. This state transition table has the input symbols on one axis and the lexer's states on the other axis. The table can be constructed by traversing the DFA from its initial state and populating it with the state, input symbol pairs represented by edges, where the state is the superseding state.

In order to lex the source code in parallel, the source code is first split into roughly equal sized parts and each is inserted into a concurrently accessible queue. A threadpool of lexers is initialized and each thread takes a part of the source code and begins to tokenize it. Each thread initialises an instance of a lexer for every possible initial state the lexer can be in, resulting in as many outputs as there are possible initial states. In other words, an individual output is a list of lexemes generated by a lexer when it begins lexing from some initial state. These outputs are collected in a concurrently accessible array until all parts of the source code are tokenized. Once the work queue is empty, the correct output from each part is chosen such that

its start state is the same as the previous chunk's finish state. Remaining outputs are discarded. This list of tokens, representing the tokenized version of the input source code, is then passed to the parser.

The following is a summary of the overall steps that need to be performed before and during lexical analysis:

1. Define the lexical grammar with a mapping between regular expressions and terminals.
2. Use to create NFA graph using Thompson's construction.
3. Convert the NFA into a DFA using powerset construction.
4. During generation, look for duplicate state transitions where the characters are the same, but states differ. This indicates that those states are potential start states of the lexer.
5. Generate a state transition table by traversing the graph.
6. Split up the source code into n chunks and put them onto a work queue.
7. These chunks are taken off the work queue by a thread pool. Each chunk is lexed for each possible start state, resulting in y number of outputs for x number of start states, per chunk n.
8. Once all chunks have been processed, the correct output from each chunk is chosen such that its start state is the same as the previous chunk's finish state. Remaining outputs are discarded.

3.2 Parser

As mentioned in section 2.6, I will implement an algorithm for parsing OPGs as shown by Barenghi et al. (2015). An OPG is a kind of context-free grammar where the right-hand side of each production rule contains at least one terminal or non-terminal and no right-hand side contains two consecutive non-terminals.

In order for it to be possible to parse the grammar in parallel, there must be a bound on the number of tokens the parser must see in order to reduce the right-hand side of a rule to its left hand side. A grammar that has such a bound is called a bounded-context grammar, as described in the literature (Grune and Jacobs, 2008). Barenghi et al. (2015) define a set of restrictions

which, when adhered to, ensure that an OPG is also a bounded-context grammar with a bound of one. They call this Fischer Normal Form (FNF) and it ensures that the parser is able to parse portions of the code without the context of what came before or after it.

An OPG is in FNF if and only if an axiom S does not occur in any right-hand side; no two rules have the same right-hand side; no rule, possibly except one with the axiom as the left-hand side, has ϵ as the right-hand side; renaming rules, i.e., those with a single non-terminal character as right-hand side, are those and only those with S as the left-hand side.

Once a grammar is in FNF, it can then be parsed by the algorithm described by Barenghi et al. (2015). A traditional linear time parser for OPG languages operates on terminals and results in a parse tree of only terminals. In order to parse this kind of grammar in parallel, a more general parsing algorithm is presented by Grune and Jacobs (2008) which allows analysing strings that contain non-terminals. Implementing this algorithm requires the construction of an operator precedence table that contains the precedence relationships between terminals in the grammar.

The input to the parser is prepared in a manner similar to the previous lexing step. The list of lexemes from the lexer is split into roughly same sized chunks that are put onto a concurrently accessible queue. A threadpool of parsers is initialized and begins working through the queue, parsing each chunk of lexemes producing a partial parse tree into a concurrently accessible output array. These partial parse trees contain the parse tree of the lexemes that were reduced, aswell as the internal stack of tokens that could not yet be reduced due to an insufficient amount of context. Once the queue is empty, the internal stacks of these partial parse trees are joined and the same parsing algorithm is used to produce the final parse tree.

The following is a summary of the overall steps that need to be performed before and during syntax analysis:

1. Read grammar from file that defines terminals, non-terminals and production rules.
2. Transform grammar into FNF as defined in Barenghi et al. (2015).
3. Build operator precedence table according to Grune and Jacobs (2008).
4. Take the lexer output from the previous step and parse each chunk using the same thread pool approach as the lexer. The parsing algorithm returns a partial parse tree and a stack of lexemes.

5. Take the outputs from each chunk and join the partial parse trees using the same parsing algorithm. This results in a parse tree of the whole input.

3.3 Semantic Analysis

In order to perform semantic analysis, the parse tree from the previous step must be transformed into an AST. This can be done by traversing the AST in post-order Depth First Traversal (DFT), reducing leaf nodes into ast nodes, thereby producing the AST bottom-up. This can be done in linear time as the parse tree resulting from the parsing step can be stored as an array whereby traversing the array from start to finish equates to a post-order traversal of the tree.

Semantic analysis is performed by traversing the AST with a pre-order DFT and keeping a record of identifiers and their metadata in a symbol table. Parallel semantic analysis can be performed by splitting the work based on nested scopes in the AST (Seshadri and Wortman, 1991). If during a pre-order DFT of the AST a new scope is encountered, the analyser starts a new thread for parsing that inner scope while passing to it the symbol table it has constructed thus far. Working from top to bottom of the AST this way ensures that the whole of the AST is analysed correctly. The drawback with this approach is that the efficacy of this parallelism is limited to the number and size of the nested scopes, which additionally makes splitting the work equally between threads a challenge.

Summary of the overall steps that need to be performed during semantic analysis:

1. Iterate over the parse tree in post-order in order to generate the AST.
2. Iterate over AST using pre-order DFT, creating a new thread and symbol table for certain nodes.
3. Check if variables are defined before they are used by checking if they exist in ancestor symbol tables.

3.4 Conclusion

The design of this compiler involves a table driven lexer and parser that can handle different languages by modifying the lexical and parsing grammars.

The lexer first converts regular expressions into a DFA and then processes its input in parallel by splitting it into chunks and processing each chunk individually. The parser employs an OPG and a bounded-context restriction to parse in parallel. This also allows it to be processed in parallel. Semantic analysis is performed by constructing an AST from the parse tree and checking variable usage using a symbol table. This is done in parallel by splitting the input in places where new scopes are created and processing each part largely in parallel.

Chapter 4 will elaborate on the details of my implementation of the design described in this chapter.

Chapter 4: Implementation

For this project I implemented the design as described in the previous chapter. It is a command line application that takes a source code file as input and performs semantic analysis on its AST. I chose to write my implementation using the Rust programming language. It is a memory safe language that leverages the LLVM backend for highly optimized code. This makes it easy to write robust and performant code. In addition, it has very good parallel programming facilities and an extensive software ecosystem that makes it ideal for this kind of project.

Due to a generic lexical and syntactic analysis implementation, as described in the previous chapter, the compiler supports parsing JSON aswell as a bespoke test language with syntax similar to Rust, shown in Listing 1 and 2.

Section 4.1 provides an explanation of the program's structure as well as some implementation details.

Section 4.3 discusses some additional, language specific code that was needed in order to support JSON and the test language.

Section 4.4 concludes with a description of the projects dependencies and their purpose in the implementation.

4.1 Structure

I will explain the implemention in the order of its execution. The program is divided into three general stages: lexical, syntactic and semantic analysis. Each stage is implemeted just it was described in the design chapter. There are significant pre-processing steps that need to happen prior to the core lexing and parsing algorithms, which can be computed at compile time,

```
fn factorial[n: int] {
    if n == 0 {
        return 1;
    }

    return (n * factorial(n - 1));
}
```

Listing 1: Factorial in the test language.

but I nevertheless chose to do at runtime. The following section begins by describing this pre-processing work that initializes key datastructures that are used later in the compiler. The subsequent two sections describe implementation details of each of the compilation stages.

4.1.1 Lexer Preprocessing

In order for the compiler to support different languages, while sharing as much code as possible, it is necessary to generate datastructures that can be used by a generic lexer and parser. Furthermore, the datastructures required for bottom-up parsing are very difficult to create without some form of automatic generation. Although a hand-written, language specific implementation has significant optimisation potential, as shown by Langdale and Lemire (2019), the focus of my implementation is to demonstrate the effectiveness of data-parallel compilation. Single threaded optimisations that leverage the language or SIMD instructions are sufficiently orthogonal to this goal to put them outside the scope of the FYP.

In order for the lexer to work, a state transition table must be obtained. This is done by reading a file that contains the mapping between the lexemes and their corresponding regular expressions. An example of the syntax can be seen in listing 3. The AST of each regular expression is then traversed and converted into one, large joint NFA graph using Thompson's construction algorithm. This is a classic algorithm that is described by Aho et al. (2006), originally attributed to Ken Thompson. It works by recursively breaking down the regex into simpler components and building an NFA for each component. The resulting NFA recognizes the same language as the original regex. For my implementation I use an external library to parse the regex into an AST. I then traverse this AST, building the NFA as described by the algorithm

```
{
  "a": 100,
  "b": {
    "x": [
      100,
      "a"
    ]
  }
}
```

Listing 2: Example of parsable JSON.

This graph is then turned into a DFA through powerset construction. For this algorithm I followed the steps outlined in wikipedia article on powerset construction (*Powerset construction*, 2023). It works by simulating an NFA and interpreting the sets of states reachable from each state in the NFA, as the states of the DFA. In other words, it neccessary to simulate the NFA and keep track of all states that the automaton could reach after seeing an input, according to its nondeterministic choices. Each set of choices is then converted into a corresponding state in the DFA.

An example NFA and DFA for the JSON grammar is shown in Figure 2 and figure 3, respectively. I implemented the graph traversals in both algorithms by keeping track of a stack of nodes that were previously encountered. The DFA can be tivially turned into a state transition table by traversing it from its initial state and taking note of each pair of states connected by a transition. The parser similarly requires an operator precedence table that must be computer generated due to its complexity.

4.1.2 Parser Preprocessing

The parser defined by Barenghi et al. (2015) is a standard operator precedence algorithm that is modified to allow the parsing of non terminals. This parser requires that the structure of the language grammar follow certain rules, as defined in Section 3.2. A preliminary validation of the grammar is performed to ensure that it conforms to these rules. This step also fixes any production rules with duplicate right-hand sides by creating new production rules with new non-terminals in a way that keeps the resulting parse tree predictable. After this validation step, the grammar is ready to be used in the parser. Using this grammar an operator precedence

table can be generated by iterating over the production rules of a grammar in order to determine the associativity of each terminal symbol.

4.1.3 Parallelisation

Each of the following compilation stages employ paralleism in the following way. The input is first split into equal sized chunks. These chunks are put onto a concurrently accessible work queue and some number of threads are started. The threads then enter a loop attempting to take work from the queue. Once a thread completes processing its work, it makes its results available to the main thread by storing it in a concurrently accessible skiplist that is common to all threads. Once all threads terminate, all the work in the workqueue will have been processed with the results stored in the pre-determined skiplist.

Each thread is initialised with a handle to access the work queue and skiplist, as well as a message passing channel to communicate with the main thread. If there is no work available, the thread checks whether it recieved a signal to terminate, otherwise the thread blocks its execution and waits to be unblocked by the main thread. Once all work on the queue has been processed, the main thread sends a message to all threads to terminate and unblocks any threads that may have been blocked. This lifecycle for managing threads ensures that threads don't waste time doing nothing and are terminated by the program rather than the operating system.

4.1.4 Lexical Analysis

The analysis begins by initialising a lexical analyzer instance for each possible state the input can be in. This set of analyzer instances is executed sequentially on a work thread, one such set per chunk of input. This is because there is no way to tell whether the analysis has begun in the middle of an identifier, string or other multi-character lexeme. The reasons for this are described in Section 2.3. The set of possible start states have been discovered during the construction of the DFA, described in Section 4.1.1.

Lexical analysis is performed by processing each character of the input and feeding it into each lexer instance. Since only the analyzers that begin at a potentially correct start state will process the input without error, if a lexer instance does encounter an error, it no longer needs to be processed.

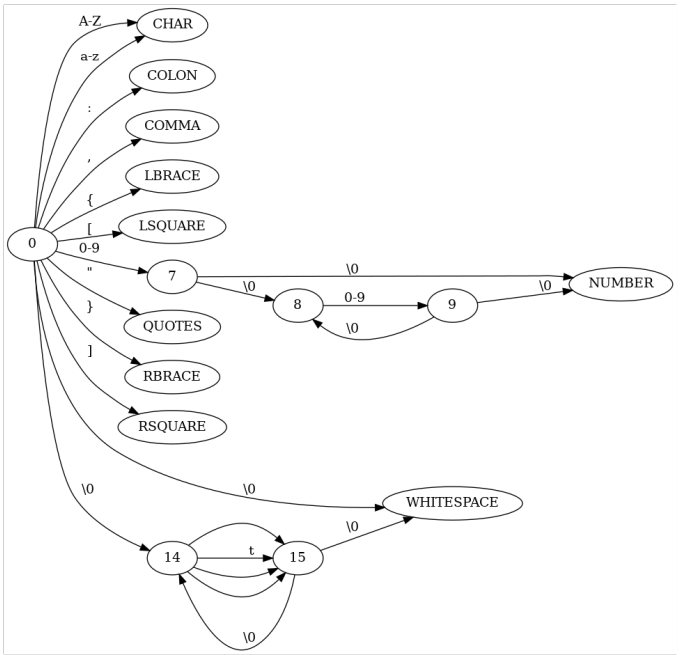


Figure 2: NFA of the JSON lexical grammar. The empty string, which is typically denoted with ϵ, is denoted here with $\backslash 0$.

Each analyzer works by the taking its current state and input symbol and using it to get a new state from the state transition table. If there is no state for the given state-symbol pair, the instance checks whether its current state is a terminal state and emits the corresponding lexeme. If there is no state it can transition to and there is no lexeme to emit, the lexer reaches an error state and no longer accepts input.

Once the input has been processed, the outputs from each group of lexer instances is placed in the output skiplist. Once the whole work queue is empty and threads have terminated, the array of outputs is joined together into one array of lexemes. This is done by iterating over the list and choosing the output's that completed analysing successfully and have the same inital state as the previous outputs final state. This array of lexemes is then passed onto the syntactic analyser.

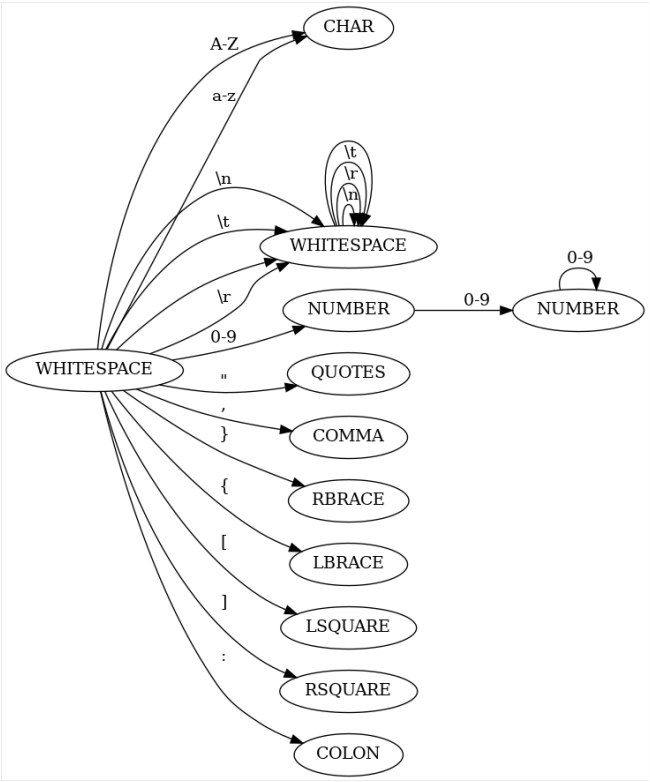
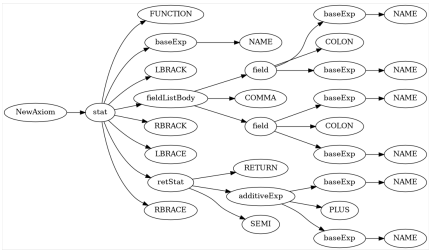


Figure 3: DFA of the JSON lexical grammar

4.1.5 Syntactic Analysis

Syntactic analysis is performed by following the algorithm described in Barenghi et al. (2015). It is a standard operator precedence parsing algorithm that is modified to allow for non terminals. Similar to the lexer, this algorithm requires two passes over the input. The first pass splits the source string into chunks. Each chunk is assigned to a separate worker and when all chunks are parsed then the outputs are recombined. Each output consists of a parse tree, possibly with leading and trailing lexemes that could not be reduced by the parser. Each two consecutive outputs are then merged such that parsing can continue on the merged whole. This continues until the parse tree is complete.




```
fn add[a: int, b: int] {
        return a + b;
}
```

Figure 4: Visualization of the parse tree on the left with the source code on the right. Terminals are capitalised and nonterminals are in camel case.

4.2 Test Language Processing

The implementation described thus far can analyse a simple language, such as JSON, without any modification. All that needs to be specified is a lexing and parsing grammar. This it not the case with the test language due to limitations in the lexing and parsing algorithms.

The syntax of a typical programming language consists of a nested list of statements. Since the parser requires a terminal between every two non terminals, any list of such statements requires some kind of non-terminal delimiter. A natural choice for this delimiter is the semicolon because it is also used for this purpose in other programming languages. An issue arises however when one considers if, while and function definition statements. Each of these statements unexpectedly requires a semicolon its end. This is very cumbersome and requires that the lexer insert semicolons between pairs, such as between right curly brackets and any lexeme that a statement can begin with.

Another issue I encountered is distinguishing between a unary and a binary minus. The parser on its own is incapable of discovering this distinction because it can potentially lack the context surrounding a minus lexeme. For example, if the expression $a-b$ were to be parsed starting from the $-$ symbol, the parser would incorrectly assume that $-$ is a unary operator with b as the operand. My solution is to emit a different lexeme during lexical analysis depending on whether any whitespace was encountered between the minus sign and the subsequent lexeme.

Due to a limitation of the regular expression parser that is used to read the lexical grammar, it is not possible to use the look-back regular expression

operator. This makes it impossible to recognize whether an input string is an identifier or a keyword of the language in the current implementation. I work around this by defining the keywords separately from the main lexical grammar and creating a second transition table from them that is then used to re-analyse any lexemes that could be a keyword of the language.

Semantic analysis of the AST checks whether all variables have been used and defined according to standard scoping rules.

4.3 Online Compiler

I decided to create an online compiler demonstration in order to visualize the compiler's outputs with a graphical user interface. This was made possible by the Rust compiler's ability to compile code into WebAssembly. The online compiler executes entirely client side without the need to run code on a remote server. WebAssembly lacks multi-threading capabilities and as such the demonstration is entirely single threaded.

4.4 Dependencies

Code dependencies in the Rust code ecosystem that are available through the cargo package manager are called crates. These crates are comparable to small, open source, third party libraries that can easily be integrated into a rust project. In this section I mention the code dependencies I've used in my implementation in order to illustrate what parts of my project were accomplished through the use of third party code.

4.4.1 Core

crossbeam - Common data structures for implementing parallel systems. This crate implements a concurrently accessible queue, skiplist and a multi-producer, multi-consumer message passing channel. All of these data structures are important for reliably sharing data across threads.

regex-syntax - A crate for parsing standard regular expression syntax. It provides a simple AST that the lexer uses to read the mappings from lexemes to regular expression patterns.

dot - A crate that makes it easier to generate graphs in the dot language. Its used to visualise the parse tree and AST.

criterion - Reliable benchmarking facilities for testing the performance of the compiler with varied sized inputs.

4.4.2 Utility or Minor Contribution

tinyrand - A lightweight implementation of random number generation. It is used for creating unique IDs for bathes of work in the work queue.

serde - Serialization and deserialization capabilities for transferring code to and from the Javascript runtime when running in the browser.

wasm-bindgen - Macros for easily creating bindings to Javascript when compiling to web assembly.

log, flexi_logger ,wasmlogger, console_error_panic_hook - Various crates for logging to files as well as to the terminal, improved formatting and correct logging when running in the browser.

simple-error - A simple library for creating errors from error messages instead of creating types for every error.

4.4.3 Optimisation

memmap - An interface for using Linux's memmap syscall. Using memmap to read large files is faster than using the standard read syscall.

bittyset - A set implementation that uses a bit vector as a backing store. This enables a compact representation as well as faster set operations in the parser where sets of terminals and non-terminals are used.

Chapter 5: Evaluation

I evaluate the performance of the lexer and parser by randomly generating JSON. The compiler was tested on a twelve core Ryzen 5 3600 CPU running Linux Mint with the 5.15.0 kernel. Figure 5 shows the speed-up achieved by increasing the number of threads used for either lexing or parsing.

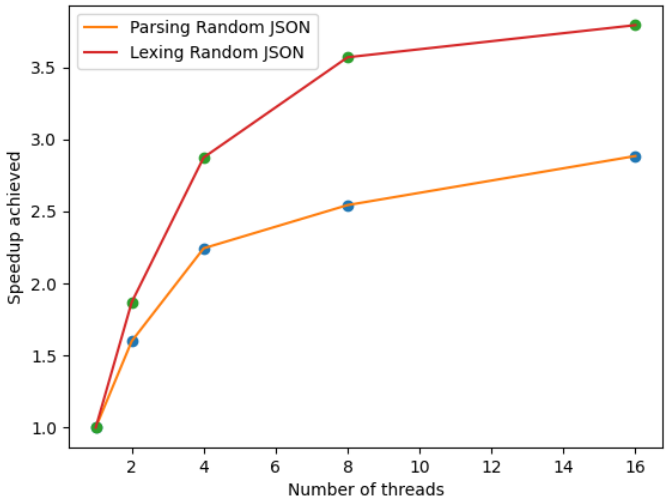


Figure 5: Times Speedup compared to the number of threads used to process 1MB of JSON.

The lexer requires that the input is processed from each possible initial state the input could possibly be in. Though this enables a simple implementation, it also results in a lot of duplicated work. The approaches discussed in the literature review aim to minimize this impact through various optimizations. These optimizations are required in order to

compete with state-of-the-art single threaded implementations however they add a significant amount of complexity to the implementation. Overall, the benefits of parallel lexing are most pronounced in areas where very large volumes of data need to be processed in parallel, such as network packet scanning Wang et al. (2019) and JSON file parsing (Li and Taura, 2023).

A potential issue in the data-parallel model that needs to be mentioned is the need to store the outputs of each compilation stage in memory. Lexing and parsing in many traditional implementations works in a streaming fashion where the lexer is only invoked when the parser requires the next token. A limitation of data-parallel computations is that the results need to joined together at the end; In the lexer this joining process cannot be parallelized. This problem didn't occur in practice due to modern computers having large amounts of available memory.

In conclusion, data-parallel computation can be leveraged to increase the performance of a compiler frontend. My implementation shows a clear increase in performance depending on the number of worker threads used. Although the benchmarks show a clear improvement with the number of threads used by the program, heavily optimized single threaded implementations such as that by Langdale and Lemire (2019) are very difficult to out compete. Future work could evaluate whether the optimizations by Langdale and Lemire (2019) can be incorporated into the proposed parallel design. Future work could also evaluate the feasibility of efficiently parsing with a larger bounded context.

Chapter 6: Conclusion

6.1 Summary

This FYP report began with an introduction to the fundamentals of sequential and parallel compilation, highlighting the traditional focus on minimizing memory use and optimizing for single-threaded performance, and juxtaposes this with the contemporary shift towards parallel processing methods to exploit the capabilities of multi-core processors. I elaborated on various parallelization methods, including SIMD, multithreading, GPGPU providing examples of their applications in compiler design.

A significant portion of this FYP delved into a literature review on methods for parallelizing compilers, focusing on lexing, parsing, and semantic analysis. I examined existing techniques, their potential for improving compiler performance, and their implications for language design. The review covers topics such as deterministic and non-deterministic finite automata, speculative simulation for lexing, parallel LR and LL parsing algorithms, and the challenges of semantic analysis in a parallelized context.

Following the literature review, I outlined the design and implementation strategies for developing a parallel compiler, including considerations for lexing and parsing. This report discusse the practical aspects of constructing a parallel compiler, emphasizing the need for carefully considering the compiler architecture such that it can efficiently handle the complexities of parallel processing.

In the evaluation chapter, I discussed the performance of my implementation as well as potential avenues for further improvements or optimisations. I conclude this report with a reflection on the research and development I've done for this FYP.

6.2 Concluding Remarks

Parallel compilation can be an effective way of speeding up the first few stages of compilation. In many systems where parsing may be a bottleneck, such as processing large amounts of JSON, there exists good evidence that parallel parsing is an effective optimisation. Further work would need to find out if the methods discussed in this FYP are compatible with the extraordinary single threaded optimisations employed by Langdale and Lemire (2019). If they are compatible then such parallel parsing techniques could push the state of the art further in JSON parsing.

Future work could also be done to investigate fully the effectiveness of parser error recovery in parallel parsers. Although there is some prior work about this (Hammond and Rayward-Smith, 1984), the research I found was written quite some time ago. Programming language research and language complexity has grown since then, which makes it worthwhile to re-assess.

```
1  CHAR = "[a-zA-Z]"
2  BOOL = "true|false"
3  NUMBER = "[0-9][0-9]*"
4  WHITESPACE = "( |\n|\t|\r)*"
5  LBRACE = "\{"
6  RBRACE = "\}"
7  LSQUARE = "\["
8  RSQUARE = "\]"
9  COMMA = ","
10 COLON = ":"
11 QUOTES = "\""
```

Listing 3: JSON lexical grammar keywords and their corresponding regular expressions

```
%nonterminal OBJECT
%nonterminal MEMBERS
%nonterminal PAIR
%nonterminal VALUE
%nonterminal STRING
%nonterminal CHARS
%nonterminal ARRAY
%nonterminal ELEMENTS

%axiom OBJECT

%terminal LBRACE
%terminal RBRACE
%terminal LSQUARE
%terminal RSQUARE
%terminal COMMA
%terminal COLON
%terminal BOOL
%terminal QUOTES
%terminal CHAR
%terminal NUMBER

%%

OBJECT : LBRACE RBRACE
       | LBRACE MEMBERS RBRACE
       ;

MEMBERS : PAIR
        | PAIR COMMA MEMBERS
        ;

PAIR : STRING COLON VALUE
     ;

VALUE : STRING
      | NUMBER
      | OBJECT
      | ARRAY
      | BOOL
      ;

STRING : QUOTES QUOTES
       | QUOTES CHARS QUOTES
       ;

CHARS : CHAR
      | CHAR CHARS
      ;

ARRAY : LSQUARE RSQUARE
      | LSQUARE ELEMENTS RSQUARE
      ;

ELEMENTS : VALUE
         | VALUE COMMA ELEMENTS
         ;
```

Listing 4: An operator precedence parsing grammar for JSON.

```
%nonterminal chunk
%nonterminal statList
%nonterminal stat
%nonterminal elseIfBlock
%nonterminal exprThenElseIfB
%nonterminal exprThen
%nonterminal name
%nonterminal retStat
%nonterminal label
%nonterminal funcName
%nonterminal nameDotList
%nonterminal varList
%nonterminal var
%nonterminal nameList
%nonterminal exprList
%nonterminal expr
%nonterminal logicalOrExp
%nonterminal logicalAndExp
%nonterminal relationalExp
%nonterminal concatExp
%nonterminal additiveExp
%nonterminal multiplicativeExp
%nonterminal unaryExp
%nonterminal caretExp
%nonterminal baseExp
%nonterminal prefixExp
%nonterminal functionCall
%nonterminal functionDef
%nonterminal parList
%nonterminal tableConstructor
%nonterminal fieldList
%nonterminal fieldListBody
%nonterminal field
%nonterminal typeExpr
%nonterminal ptrTypeStart

%axiom chunk

%terminal ENDFILE
%terminal RETURN
%terminal SEMI
%terminal COLON
%terminal COLON2
%terminal DOT
%terminal DOT3
%terminal COMMA
%terminal LBRACK
%terminal RBRACK
%terminal LBRACE
%terminal RBRACE
%terminal LPAREN
%terminal RPAREN
%terminal EQ
%terminal BREAK
%terminal GOTO
```

```
56  %terminal DO
57  %terminal END
58  %terminal WHILE
59  %terminal REPEAT
60  %terminal UNTIL
61  %terminal IF
62  %terminal THEN
63  %terminal ELSEIF
64  %terminal ELSE
65  %terminal FOR
66  %terminal IN
67  %terminal FUNCTION
68  %terminal LET
69  %terminal NIL
70  %terminal FALSE
71  %terminal TRUE
72  %terminal NUMBER
73  %terminal STRING
74  %terminal NAME
75  %terminal PLUS
76  %terminal MINUS
77  %terminal ASTERISK
78  %terminal DIVIDE
79  %terminal CARET
80  %terminal PERCENT
81  %terminal DOT2
82  %terminal LT
83  %terminal GT
84  %terminal LTEQ
85  %terminal GTEQ
86  %terminal EQDOUBLE
87  %terminal NEQ
88  %terminal AND
89  %terminal OR
90  %terminal NOT
91  %terminal UMINUS
92  %terminal SHARP
93  %terminal SEMIFIELD
94  %terminal EQ
95  %terminal QUESTIONMARK
96  %terminal STRUCT
97  %terminal COMMENT
98
99  %%
100
101 chunk : statList
102         ;
103
104 statList : stat
105         | SEMI
106         | stat SEMI
107         | statList SEMI stat
108         | statList SEMI
109         ;
110
111 stat : functionCall
```

```
112         | retStat
113         | LBRACE statList RBRACE
114         | LBRACE RBRACE
115         | WHILE expr LBRACE statList RBRACE
116         | WHILE expr LBRACE RBRACE
117         | IF expr LBRACE statList RBRACE
118         | IF expr LBRACE RBRACE
119         | IF expr LBRACE statList RBRACE elseIfBlock
120         | IF expr LBRACE RBRACE elseIfBlock
121         | STRUCT baseExp LBRACE RBRACE
122         | STRUCT baseExp LBRACE fieldList RBRACE
123         | FUNCTION baseExp LBRACK RBRACK LBRACE statList RBRACE
124         | FUNCTION baseExp LBRACK fieldList RBRACK LBRACE statList RBRACE
125         | FUNCTION baseExp LBRACK fieldList RBRACK LBRACE RBRACE
126         | FUNCTION baseExp LBRACK baseExp RBRACK LBRACE statList RBRACE
127         | FUNCTION baseExp LBRACK baseExp RBRACK LBRACE RBRACE
128         | FUNCTION baseExp LBRACK RBRACK LBRACE RBRACE
129         | FOR nameList IN exprList LBRACE statList RBRACE
130         | FOR nameList IN exprList LBRACE RBRACE
131         | LET baseExp EQ expr
132         | LET baseExp
133         ;
134
135 functionCall : baseExp LPAREN exprList RPAREN
136         | baseExp LPAREN expr RPAREN
137         | baseExp LPAREN RPAREN
138         ;
139
140 retStat : RETURN SEMI
141         | RETURN exprList SEMI
142         | RETURN expr SEMI
143         | RETURN
144         | RETURN exprList
145         | RETURN expr
146         ;
147
148 elseIfBlock : ELSEIF expr LBRACE statList RBRACE
149         | ELSEIF expr LBRACE RBRACE
150         | ELSEIF expr LBRACE RBRACE elseIfBlock
151         | ELSEIF expr LBRACE statList RBRACE elseIfBlock
152         | ELSE LBRACE RBRACE
153         | ELSE LBRACE statList RBRACE
154         ;
155
156 exprList : expr COMMA expr
157         | exprList COMMA expr
158         ;
159
160 expr : logicalOrExp
161         ;
162
163 logicalOrExp : logicalAndExp
164         | logicalOrExp OR logicalAndExp
165         ;
166
167 logicalAndExp : relationalExp
```

```
168         | logicalAndExp AND relationalExp
169         ;
170
171 relationalExp : additiveExp
172         | relationalExp LT additiveExp
173         | relationalExp GT additiveExp
174         | relationalExp LTEQ additiveExp
175         | relationalExp GTEQ additiveExp
176         | relationalExp NEQ additiveExp
177         | relationalExp EQDOUBLE additiveExp
178         ;
179
180 additiveExp : multiplicativeExp
181         | additiveExp PLUS multiplicativeExp
182         | additiveExp MINUS multiplicativeExp
183         ;
184
185 multiplicativeExp : unaryExp
186         | multiplicativeExp ASTERISK unaryExp
187         | multiplicativeExp DIVIDE unaryExp
188         | multiplicativeExp PERCENT unaryExp
189         ;
190
191 unaryExp : caretExp
192         | NOT unaryExp
193         | SHARP unaryExp
194         | UMINUS unaryExp
195         ;
196
197 caretExp : baseExp
198         | baseExp CARET caretExp
199         ;
200
201 baseExp : NIL
202         | FALSE
203         | TRUE
204         | NUMBER
205         | STRING
206         | NAME
207         | prefixExp
208         ;
209
210 prefixExp : var
211         | functionCall
212         | LPAREN expr RPAREN
213         ;
214
215 fieldList : fieldListBody
216         | fieldListBody COMMA
217         ;
218
219 fieldListBody : field
220         | fieldListBody COMMA field
221         ;
222
223 field : baseExp COLON baseExp
```

```
224         ;
225
226 var : baseExp DOT baseExp
227         | prefixExp DOT baseExp
228         ;
229
230 varList : var COMMA var
231         | varList COMMA var
232         ;
233
234 funcName : nameDotList
235         | nameDotList COLON baseExp
236         ;
237
238 nameDotList : baseExp DOT baseExp
239         | nameDotList DOT baseExp
240         ;
```

Listing 5: An operator precedence parsing grammar for the test language.

References

Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. (2006), *Compilers: Principles, Techniques, and Tools (2nd Edition)*, Addison-Wesley Longman Publishing Co., Inc., USA.

ARM Architecture Reference Manual Thumb-2 Supplement (n.d.).
URL: https://developer.arm.com/documentation/ddi0308/d/Thumb-Instructions/Alphabetical-list-of-Thumb-instructions/UQADD8?lang=en

Barenghi, A., Crespi Reghizzi, S., Mandrioli, D., Panella, F. and Pradella, M. (2015), 'Parallel parsing made practical', *Science of Computer Programming* **112**, 195–226.
URL: https://linkinghub.elsevier.com/retrieve/pii/S0167642315002610

Bernecky, R. (2003), An SPMD/SIMD Parallel Tokenizer for APL, *in* 'Proceedings of the 2003 Conference on APL: Stretching the Mind', APL '03, Association for Computing Machinery, New York, NY, USA, pp. 21–32. event-place: San Diego, California.
URL: https://doi.org/10.1145/882067.882071

Clarke, G. and Barnard, D. T. (1993), 'Error handling in a parallel LR substring parser', *Computer Languages* **19**(4), 247–259.
URL: https://www.sciencedirect.com/science/article/pii/009605519390010X

Difficulties improving C++ (n.d.).
URL: https://github.com/carbon-language/carbon-lang/blob/456d16525285c337c6b239ea4fcbfad017905 2c1b/docs/project/difficulties_improving_cpp.md

Gross, T., Sobel, A. and Zolg, M. (1989), Parallel Compilation for a Parallel Machine, *in* 'Proceedings of the ACM SIGPLAN 1989 Conference on Programming Language Design and Implementation', PLDI '89, Association for Computing Machinery, New York, NY, USA, pp. 91–100. event-place: Portland, Oregon, USA.
URL: https://doi.org/10.1145/73141.74826

Grune, D. and Jacobs, C. J. H. (2008), *Parsing Techniques*, Monographs in Computer Science, Springer New York, New York, NY.
URL: *http://link.springer.com/10.1007/978-0-387-68954-8*

Hall, M., Padua, D. and Pingali, K. (2009), 'Compiler Research: The next 50 Years', *Commun. ACM* **52**(2), 60–67. Place: New York, NY, USA Publisher: Association for Computing Machinery.
URL: https://doi.org/10.1145/1461928.1461946

Hammond, K. and Rayward-Smith, V. J. (1984), 'A survey on syntactic error recovery and repair', *Computer Languages* **9**(1), 51–67.
URL: *https://www.sciencedirect.com/science/article/pii/0096055184900122*

Hillis, W. D. and Steele, G. L. (1986), 'Data Parallel Algorithms', *Commun. ACM* **29**(12), 1170–1183. Place: New York, NY, USA Publisher: Association for Computing Machinery.
URL: https://doi.org/10.1145/7902.7903

Holub, J. and Štekr, S. (2009), On Parallel Implementations of Deterministic Finite Automata, *in* S. Maneth, ed., 'Implementation and Application of Automata', Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 54–64.

Howard, P. G. and Vitter, J. S. (1996), 'Parallel lossless image compression using Huffman and arithmetic coding', *Information Processing Letters* **59**(2), 65–73.
URL: https://www.sciencedirect.com/science/article/pii/0020019096000907

Hutchison, L. A. D. (2020), 'Pika parsing: reformulating packrat parsing as a dynamic programming algorithm solves the left recursion and error recovery problems'. _eprint: 2005.06444.

Ko, Y., Jung, M., Han, Y.-S. and Burgstaller, B. (2012), 'A Speculative Parallel DFA Membership Test for Multicore , SIMD and Cloud Computing Environments', *International Journal of Parallel Programming* **42**.

Langdale, G. and Lemire, D. (2019), 'Parsing gigabytes of JSON per second', *The VLDB Journal* **28**(6), 941–960. Publisher: Springer Science and Business Media LLC.
URL: *http://dx.doi.org/10.1007/s00778-019-00578-5*

Li, L., Sato, S., Liu, Q. and Taura, K. (2021), Plex: Scaling Parallel Lexing with Backtrack-Free Prescanning, *in* '2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)', IEEE, Portland, OR, USA, pp. 693–702.
URL: *https://ieeexplore.ieee.org/document/9460518/*

Li, L. and Taura, K. (2023), Associative Operator Precedence Parsing: A Method To Increase Data Parsing Parallelism, *in* 'Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region', ACM, Singapore Singapore, pp. 75–87.
URL: *https://dl.acm.org/doi/10.1145/3578178.3578233*

Luchaup, D., Smith, R., Estan, C. and Jha, S. (2009), Multi-byte Regular Expression Matching with Speculation, *in* E. Kirda, S. Jha and D. Balzarotti, eds, 'Recent Advances in Intrusion Detection', Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 284–303.

Luchaup, D., Smith, R., Estan, C. and Jha, S. (2011), 'Speculative Parallel Pattern Matching', *Trans. Info. For. Sec.* **6**(2), 438–451. Publisher: IEEE Press.
URL: *https://doi.org/10.1109/TIFS.2011.2112647*

Mark Thierry Vandevoorde (1988), Parallel Compilation on a Tightly Coupled Multiprocessor, Technical report.
URL: *https://www.hpl.hp.com/techreports/Compaq-DEC/SRC-RR-26.pdf*

Medeiros, S. and Mascarenhas, F. (2018), Syntax error recovery in parsing expression grammars, *in* 'Proceedings of the 33rd Annual ACM Symposium on Applied Computing', ACM.

Mickunas, M. D. and Schell, R. M. (1978), Parallel Compilation In A Multiprocessor Environment (Extended Abstract), *in* 'Proceedings of the 1978 Annual Conference', ACM '78, Association for Computing Machinery, New York, NY, USA, pp. 241–246. event-place: Washington, D.C., USA.
URL: *https://doi.org/10.1145/800127.804105*

Most vexing parse (2022). Page Version ID: 1108385094.
URL: https://en.wikipedia.org/w/index.php?title=Most_vexing_parse&oldid=1108385094

Mytkowicz, T., Musuvathi, M. and Schulte, W. (2014), Data-Parallel Finite-State Machines, *in* 'Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems', ASPLOS '14, Association for Computing Machinery, New York, NY, USA, pp. 529–542. event-place: Salt Lake City, Utah, USA.
URL: https://doi.org/10.1145/2541940.2541988

Nicholas Nethercote (2023), 'Faster compilation with the parallel front-end in nightly | Rust Blog'.
URL: https://blog.rust-lang.org/2023/11/09/parallel-rustc.html

Powerset construction (2023). Page Version ID: 1146637751.
URL: https://en.wikipedia.org/w/index.php?title=Powerset_construction&oldid=1146637751

Roesch, M. (1999), Snort - Lightweight Intrusion Detection for Networks, *in* 'Proceedings of the 13th USENIX Conference on System Administration', LISA '99, USENIX Association, USA, pp. 229–238. event-place: Seattle, Washington.

Rui Ueyama (n.d.), 'Design and implementation of mold'.
URL: https://github.com/rui314/mold/blob/8131528305 3c2c9db294a042900f26634 9c8d2dd/docs/design.md

Scarpazza, D. P. and Russell, G. F. (2009), High-Performance Regular Expression Scanning on the Cell/ B.E. Processor, *in* 'Proceedings of the 23rd International Conference on Supercomputing', ICS '09, Association for Computing Machinery, New York, NY, USA, pp. 14–25. event-place: Yorktown Heights, NY, USA.
URL: https://doi.org/10.1145/1542275.1542284

Scott, M. (2015), *Programming Language Pragmatics*, Elsevier Science.
URL: https://books.google.ie/books?id=jM-cBAAAQBAJ

Seshadri, V. and Wortman, D. B. (1991), 'An investigation into concurrent semantic analysis', *Software: Practice and Experience* **21**(12), 1323–1348.
URL: *https://onlinelibrary.wiley.com/doi/10.1002/spe.4380211205*

Shalf, J. (2020), 'The future of computing beyond Moore's Law', *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* **378**(2166), 20190061.
URL: https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061

Sin'ya, R., Matsuzaki, K. and Sassa, M. (2013), Simultaneous Finite Automata: An Efficient Data-Parallel Model for Regular Expression Matching, *in* 'Proceedings of the International Conference on Parallel Processing'.

Skrzypczak, P. (2012), 'Parallel parsing of context-free grammars'.

Stefan, G. M. (2021), Let's consider Moore's law in its entirety, *in* '2021 International Semiconductor Conference (CAS)', pp. 3–10.

Vagner, L. and Melichar, B. (2007), 'Parallel LL parsing', *Acta Informatica* **44**(1), 1–21.
URL: *http://link.springer.com/10.1007/s00236-006-0031-y*

Voetter, R. F., Huijben, M. and Rietveld, K. F. D. (2022), Compilation on the GPU?: a feasibility study, *in* 'Proceedings of the 19th ACM International Conference on Computing Frontiers', ACM, Turin Italy, pp. 230–236.
URL: https://dl.acm.org/doi/10.1145/3528416.3530249

Wang, X., Hong, Y., Chang, H., Park, K., Langdale, G., Hu, J. and Zhu, H. (2019), Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs, *in* '16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)', USENIX Association, Boston, MA, pp. 631–648.
URL: https://www.usenix.org/conference/nsdi19/presentation/wang-xiang

Ward-Foxton, S. (2023), 'ROCm Is AMD's No. 1 Priority, Exec Says', *EE Times* .
URL: https://www.eetimes.com/rocm-is-amds-no-1-priority-exec-says/

Williams, R. S. (2017), 'What's Next? [The end of Moore's law]', *Computing in Science & Engineering* **19**(2), 7–13.

Zhao, Z. and Shen, X. (2015), 'On-the-Fly Principled Speculation for FSM Parallelization', *SIGPLAN Not.* **50**(4), 619–630. Place: New York, NY, USA Publisher: Association for Computing Machinery.
URL: *https://doi.org/10.1145/2775054.2694369*